

INTERNATIONAL UNIVERSITY OF EAST AFRICA
FACULTY OF SCIENCE AND TECHNOLOGY



AfriLang

A Mini Compiler Front End for Community Health Center Records

Mini Compiler Construction Project — Scenario E: Community Health Center

Student Name: **Mbainaissem Dougt**

Registration Number: 23/1516/BSCS-S

Course/Module: CSC3200- Compiler Construction

Tools Used: Java, JFlex, Java CUP

Date: July 2026

1. Selected African Scenario

The scenario chosen for this project is Scenario E: Community Health Centre. Community health centers across many African settings keep simple digital records of patients, their vital signs, scheduled appointments, and their admission status. A lightweight domain-specific language reduces the training burden on front-desk and clinical staff compared to a general-purpose programming language, since the vocabulary maps directly onto the tasks they already perform on paper.

AfriLang lets a clinic worker type short, readable statements describing what happened during a visit — registering a patient, recording a temperature, booking a follow-up appointment, admitting or discharging someone and have the system check that the instructions are well formed before they are acted on.

2. Language Description

AfriLang is a small, statement-oriented language. Every statement ends with a semicolon except a conditional block, which is delimited by braces. The language purpose is narrow by design: it exists to validate structured clinic transactions, not to be a general programming language.

2.1 Supported Keywords

patient, admit, discharge, temperature, weight, appointment, show, age, on, if

2.2 Operators and Delimiters

Assignment: = Arithmetic: + - * / Comparison: > < >= <= == != Delimiters: ; () { }

2.3 Identifier Rules

An identifier must begin with a letter and may then contain any combination of letters, digits, and underscores (e.g. Musa, patient_1, temperature_reading). Identifiers are case-sensitive and are distinct from keywords.

2.4 Sample Valid Program

```
// Register a patient and take a temperature reading
patient Musa age = 35;
temperature Musa = 38.5;
appointment Musa on "2026-07-16";
show patient Musa;
```

3. Token Table

Category	Examples	Pattern / Rule
Keyword	patient, admit, discharge, temperature, weight, appointment, show, age, on, if	Exact reserved word match, checked before identifiers
Identifier	Musa, Amina, temperature_reading	letter (letter digit _)*
Integer	35, 100000	digit+
Decimal	38.5, 2500.50	digit+ . digit+
String	"2026-07-16"	" (any char except " or newline)* "
Assignment	=	single character
Arithmetic	+ - * /	single character each
Comparison	> < >= <= == !=	one or two characters
Terminator	;	single character
Delimiters	() { }	single character each
Comment	// remaining line text	// followed by any characters up to end of line
Whitespace	space, tab, newline	ignored, not returned as a token

Mbainaissem Dought

4. Grammar Productions

The grammar below follows the structure suggested in the project brief, adapted to the Health Centre scenario's statement types.

```

program      ::= statement_list
statement_list ::= statement_list statement | statement
statement    ::= declaration ';'
              | assignment ';'
              | transaction ';'
              | output_statement ';'
              | conditional_statement
declaration  ::= 'patient' IDENTIFIER 'age' '=' INTEGER
assignment   ::= 'temperature' IDENTIFIER '=' expression
              | 'weight' IDENTIFIER '=' expression
              | IDENTIFIER '=' expression
transaction  ::= 'admit' IDENTIFIER
              | 'discharge' IDENTIFIER
              | 'appointment' IDENTIFIER 'on' STRING
output_statement ::= 'show' 'patient' IDENTIFIER
              | 'show' 'temperature' IDENTIFIER
              | 'show' 'weight' IDENTIFIER
              | 'show' IDENTIFIER
conditional_stmt ::= 'if' condition '{' statement_list '}'
condition        ::= expression comparison_operator expression
expression       ::= expression '+' term | expression '-' term | term
term             ::= term '*' factor | term '/' factor | factor
factor          ::= INTEGER | DECIMAL | IDENTIFIER | '(' expression ')'

```

Multiplication and division bind tighter than addition and subtraction, and parentheses override both, matching normal arithmetic precedence. This is enforced in AfriLang.cup with CUP's precedence declarations for the PLUS/MINUS and TIMES/DIVIDE terminal groups.

5. Scanner Explanation (AfriLang.flex)

The scanner is defined in AfriLang.flex and generates a Lexer class through JFlex. It is declared with %cup so that its return type and token wrapping match what the CUP-generated parser expects, and %line / %column so that every token carries its line and column position forward.

- Comments and whitespace are matched by their own rules and produce no token, so they are effectively discarded before parsing begins.
- Keyword rules are listed before the general identifier rule; JFlex's longest-match-then-first-listed-rule behaviour means a word like 'patient' is captured as the PATIENT keyword rather than a generic identifier.
- Decimal numbers are matched before integers are considered complete, so '38.5' is read as one DECIMAL token rather than an INTEGER followed by a stray '.5'.
- Every recognized token is wrapped through a ComplexSymbolFactory helper so that CUP receives both the token's semantic value (e.g. the parsed number or identifier text) and its source location.
- Any character that matches none of the rules above (the catch-all '.' rule) is reported immediately as a lexical error with its line and column, and scanning continues so that later errors in the same file can also be found in one pass.

6. Parser Explanation (AfriLang.cup)

The parser is defined in AfriLang.cup. It declares one terminal per token type produced by the scanner and one non-terminal per grammar rule described in Section 4. Each production carries an embedded action that appends a short, human-readable description of the statement to a running list, which is what Main.java prints as the 'Parsed statements' report on success.

Arithmetic precedence is declared explicitly with 'precedence left PLUS, MINUS;' followed by 'precedence left TIMES, DIVIDE;', which resolves the shift/reduce ambiguity in the expression grammar the same way ordinary arithmetic does. Parenthesized sub-expressions are handled by a dedicated factor production, which lets them override the default precedence.

Syntax errors are reported through an overridden syntax_error method, which prints the offending line number and the unexpected token rather than letting CUP throw a raw exception. unrecovered_syntax_error is also overridden so a single bad statement does not abort the whole run before the error message has been printed.

7. Test Results

Six test programs were prepared under tests/, covering the required mix of valid and invalid input. Expected results, derived by tracing each program against the grammar in Section 4, are summarized below; actual console output/screenshots should be captured after running the real JFlex/CUP build (see Appendix A) and inserted into the screenshots/ folder.

Test File	Purpose	Expected Result
valid_test1.afri	Register patient, record temperature, book appointment, query patient	Parses successfully; 4 statements reported
valid_test2.afri	Admission, weight recording, arithmetic assignment	Parses successfully; 4 statements reported
valid_test3.afri	Conditional block with comparison and nested statements	Parses successfully; conditional block plus nested statements reported
lexical_error.afri	Illegal character '@' inside a decimal literal	1 lexical error reported; compilation fails
syntax_error.afri	Missing semicolon after a declaration	1 syntax error reported; compilation fails
multiple_errors.afri	Combination of a missing semicolon and an illegal character	2+ errors reported in a single pass; compilation fails

8. Challenges Encountered

- Deciding how much semantic detail (e.g. resolving identifiers to stored values) belongs in a compiler front end versus a later interpretation stage this project intentionally stops at recognising and reporting valid structure, not executing the transactions.
- Getting statement-level error recovery right so that one missing semicolon does not stop the whole file from being checked, while still reporting an accurate line number for the actual problem.
- Choosing keyword-prefixed statement forms (e.g. 'show patient X' vs 'show X') that stay unambiguous for a single-token lookahead LALR parser.
- This particular build environment had no internet access and no JDK, only a Java runtime, so the JFlex/CUP generation and compilation steps could not be executed in that environment and had to be completed on a separate machine with the full toolchain.

9. Conclusion

AfriLang demonstrates the standard compiler front-end pipeline — lexical analysis with JFlex, syntax analysis with Java CUP, and diagnostic reporting applied to a small, realistic African health-centre record-keeping task. The resulting scanner and parser recognize patient declarations, vital-sign updates, appointments, admissions/discharges, output queries and conditional blocks, enforce normal arithmetic precedence, and report both lexical and syntax errors with source locations, meeting the minimum requirements set out in the project brief.

Appendix A: How to Compile and Run AfriLang

These steps assume a computer with internet access and a full JDK (not just a JRE).

A.1 Recommended environment

- IDE: IntelliJ IDEA (Community Edition is free) has good support for running plain javac/java commands from its integrated terminal, and JFlex/CUP plugins exist for syntax highlighting of .flex/.cup files.
- Alternative: Visual Studio Code with the 'Extension Pack for Java', lighter weight, works fine since the actual build is driven from the terminal, not the IDE's own build system.
- Either way, the JFlex and CUP generation steps are run from a terminal/command line, not from IDE menus no IDE has built-in JFlex/CUP support, so the terminal commands below are required regardless of which editor is used.

A.2 Prerequisites

- A JDK, version 11 or later (java -version and javac -version should both work).
- jflex-1.9.0.jar, downloaded from <https://jflex.de/download.html>
- java-cup-11b.jar and java-cup-11b-runtime.jar, downloaded from <https://www2.cs.tum.edu/projects/cup/>

A.3 Step-by-step

1. Open a terminal in the project's root folder (the one containing src/, tests/, generated/).

```
mkdir -p generated out
```

2. Generate the scanner from the JFlex specification. This reads src/AfriLang.flex and writes Lexer.java into generated/.

```
java -jar jflex-1.8.2.jar -d generated src/AfriLang.flex
```

3. Generate the parser from the CUP specification. This reads src/AfriLang.cup and writes parser.java and sym.java into generated/.

```
java -jar java-cup-11b.jar -parser parser -symbols sym -destdir generated src/AfriLang.cup
```

4. Compile everything together — the hand-written driver plus the generated scanner/parser against the CUP runtime jar.

```
javac -cp java-cup-11b-runtime.jar -d out src/Main.java generated/*.java
```

5. Run the compiled program against any test file, passing the CUP runtime jar on the classpath alongside the compiled classes.

```
java -cp "out:java-cup-11b-runtime.jar" Main tests/valid_test1.afri
```

(On Windows, replace the ':' classpath separator in the command above with ';', e.g. "out;java-cup-11b-runtime.jar".)

6. Repeat step 5 for each of the six files in tests/, and copy or screenshot each run's console output into screenshots/ for submission.

A.4 If you would rather work inside an IDE project

Create a new plain Java project (not Maven/Gradle, to keep this simple), then:

- Add `java-cup-11b-runtime.jar` to the project's libraries/classpath (IntelliJ: File > Project Structure > Libraries; VS Code: add it to `referencedLibraries` in `.vscode/settings.json`).
- Run the two generation commands from Step 2 and 3 above in the IDE's built-in terminal — they are the same regardless of IDE.
- Mark `generated/` as a source root (IntelliJ: right-click the folder > Mark Directory as > Sources Root) so the IDE picks up `Lexer.java`, `parser.java` and `sym.java` alongside `src/Main.java`.
- Right-click `Main.java` and Run, supplying the test file path as a program argument (IntelliJ: Run > Edit Configurations > Program arguments).

A.5 Quick check without JFlex/CUP installed yet

The `reference/` folder contains a small hand-written Java lexer and parser that follow the exact same token and grammar rules as `AfriLang.flex/AfriLang.cup`, with no JFlex/CUP dependency. It compiles with a plain `javac` and is useful for confirming the grammar's logic while you are still setting up the real toolchain but it does not count as your JFlex/CUP submission.

```
cd reference
javac *.java
java RefMain ../tests/valid_test1.afri
```